Lecture 3

Thursday January 22, 2026

UNIX C Programming

- Lecture 3 slides cover basics of C on the UNIX system
- Slides cover basic commands with the vi editor
- Quick demo on using vi **(watch recording of lecture)**
  - Press `a` to enter insert mode
  - `escape` `:wq` to save and exit
- Emacs and nano are other options to use

Files and Directories

- review how to navigate a Linux CLI
- `ll` will show you information on your file system
- `d` means its a directory
- `-` means its a file
- all files have permissions that determine who can **read (r)**, **write (w)**, and **execute (x)** a file
- Three groups of three:
  - **User**
  - **Group**
  - **Others**
- 777 permission is full open permission for everyone also read as 111, 111, 111 which is 7 in binary for each group
  - Bad idea from a security stand point
- `chmod` can be used to alter the permissions
  - e.x. `chmod 760 text.txt`

Processes

- Every thing you run on a computer is a **process** while it is being executed
- `ps -a` all process list
- `top` works similar to the task manager
  - shows all running processes
  - most are running under root which is the super user special privilege

    - you can use `sudo` or `su` to run with root privilege that ends after the command is executed
- `kill n` kill process with id=n

Basic C

- Several slides detail basic C programs such as hello world
- Review the slides
- C++ is a super set over C
- Look into how to format the print output
- Reading from `stdin` and `get`
- arrays and pointers work the same
- OS doesn't work without pointers
- Strings operate as an array of char
- Comparison operators
- Loops
- Functions
- Memory manipulation refers to pointers
- Defining pointers will be important
- Parameter passing in C works with pointers
- Structures are basically classes without functions
    - You need to define them yourself to use
- enums, static and global variables
- Dynamic memory allocation with malloc

Code example in pintos VM

- Watch lecture recording at home and follow along
- In previous lecture we ran multiple versions of the same infinite program
- The CPU assigned random time intervals to each so they could all run.
- A single CPU does not allow parallelization so it uses the time sharing to make it appear that it is running concurrently to the user
- Review how `int main (int argc, char argv[])` works with CLI input
- Program **(threads.c)** creates two threads let them do their work then join them
- `gcc -0 threads threads.c -pthread -Wall` to create the executable file
- `./threads 5` command to run the program with 5 loops that each thread will execute
    - With the input of 5 the final value would be 10 because both of the threads count to 5
- `./threads 1000000` will break the program and it will return random numbers caused by a **race condition**

- 5 is a small enough number that it doesn't run into the issue but larger values cause issue with which thread is able to update the count
- We will learn how to deal with this in the course